

Статический полиморфизм в Go

Дженерики: type parameters, constraints, type sets

Пучкин Станислав (СПбГУ)

C++ Семинар

8 апреля 2026 г.

Зачем нужны дженерики

```
1 // До Go 1.18: interface{} - небезопасный "any"
2 func MinOld(a, b interface{}) interface{} {
3     if a.(int) < b.(int) { return a } // panic если не int!
4     return b
5 }
6 MinOld("x", 42) // компилируется, падает в runtime
7
8 // Go 1.18+: type parameter с constraint
9 func Min[T cmp.Ordered](a, b T) T {
10     if a < b { return a }
11     return b
12 }
13 Min(3, 5) // T=int, проверено компилятором
14 Min("b", "a") // T=string, проверено компилятором
15 // Min("x", 1) // ошибка компиляции: string != int
16
17 // C++ templates: duck typing, ошибка при инстанцировании
18 // Go generics: явный constraint, ошибка при определении
```

```
1 func Swap[T any](a, b T) (T, T) { return b, a }
2
3 // Вывод типа: T выводится из аргументов
4 x, y := Swap(1, 2)           // T = int
5 a, b := Swap("hi", "go")    // T = string
6
7 // Явное указание всегда допустимо:
8 p, q := Swap[float64](3.0, 1.0)
9
10 // Для generic типов - тип аргумент - ОБЯЗАТЕЛЕН:
11 var s Stack[int]           // OK
12 // var t Stack             // ошибка: missing type argument
13
14 // Сравнение: C++ имеет CTAD для типов
15 // std::pair p(1, 2.0) → pair<int, double> // работает
16 // В Go для типов - нет: Stack{...}        // ошибка
```

Constraints: type sets и underlying types

```
1 // Constraint - интерфейс как множество допустимых типов
2 type Numeric interface {
3     int | int32 | int64 | float32 | float64
4 }
5 func Sum[T Numeric](s []T) T {
6     var r T; for _, v := range s { r += v }; return r
7 }
8
9 // ~ : включает именованные типы с тем же underlying type
10 type Celsius float64
11 type Temperature interface{ ~float64 }
12
13 func ToKelvin[T Temperature](c T) float64 {
14     return float64(c) + 273.15
15 }
16 ToKelvin(Celsius(0))    // OK: underlying type = float64
17 // ToKelvin(42)         // ошибка: int not in Temperature
18
19 // Объединение нескольких underlying types:
20 type Real interface{ ~int | ~int64 | ~float64 }
```

any и comparable

```
1 // any = interface{} - constraint без ограничений
2 func Map[T any, R any](s []T, f func(T) R) []R {
3     result := make([]R, len(s))
4     for i, v := range s { result[i] = f(v) }
5     return result
6 }
7
8 // comparable: поддерживает == и !=
9 // comparable: int, string, struct с полями comparable
10 // НЕ comparable: slice, map, func
11 func Contains[T comparable](s []T, v T) bool {
12     for _, x := range s {
13         if x == v { return true }
14     }
15     return false
16 }
17
18 sq := Map([]int{1,2,3}, func(x int) int { return x*x })
19 fmt.Println(Contains(sq, 4)) // true
20 // Contains([][]int{{}}, []int{}) // ошибка: not comparable
```

```
1  type Stack[T any] struct{ data []T }
2
3  func (s *Stack[T]) Push(v T) {
4      s.data = append(s.data, v)
5  }
6  func (s *Stack[T]) Pop() (T, bool) {
7      if len(s.data) == 0 {
8          var zero T; return zero, false
9      }
10     n := len(s.data) - 1
11     v, s.data = s.data[n], s.data[:n]
12     return v, true
13 }
14
15 var s Stack[int]
16 s.Push(1); s.Push(2)
17 v, ok := s.Pop() // v=2, ok=true
18
19 // Методы получают T от типа; свой T у метода невозможен:
20 // func (s *Stack[T]) As[R any]() R {...} // ошибка!
```

Перегрузка функций в Go: её нет

```
1 // В Go нет перегрузки - одно имя, одна функция
2 // func Min(a, b int) int           { ... } // OK
3 // func Min(a, b float64) float64 { ... } // ошибка!
4
5 // До дженериков: отдельное имя для каждого типа
6 func MinInt(a, b int) int {
7     if a < b { return a }; return b
8 }
9 func MinFloat64(a, b float64) float64 {
10     if a < b { return a }; return b
11 }
12
13 // Дженерики: одна реализация для всех ordered типов
14 func Min[T cmp.Ordered](a, b T) T {
15     if a < b { return a }
16     return b
17 }
18 Min(3, 5); Min(3.14, 2.72); Min("b", "a")
19
20 // НО: одно тело - нельзя разное поведение для разных T
21 // нет(специализации; в C++: template<> void f<int>(){})
```

```
1 // 1. Методы не могут иметь свои type параметры -
2 type Box[T any] struct{ v T }
3 // func (b Box[T]) As[R any]() R {...} // ошибка!
4 // C++: template<typename R> auto as() → OK
5
6 // 2. Нет специализации одно( тело для всех T)
7 // C++: template<> void f<int>() { /* особый код */ }
8 func PrintAll[T any](s []T) {
9     for _, v := range s { fmt.Println(v) }
10 }
11
12 // 3. Операторы - только через constraint
13 // C++: a+b проверяется при инстанцировании шаблона
14 // Go: проверяется при определении функции
15 type Addable interface{ int | float64 }
16 func Add[T Addable](a, b T) T { return a + b }
17 // func Bad[T any](a, b T) T { return a + b } // ошибка!
```